GOLD: Inheritance (Mon Sep 28)

Example: Person

A university personnel program

Suppose you have a job in Purdue HR. The university needs to track basic data.

Start the obvious way:

```
class Main {
    void main() {
        Person p = new Person("Ada Lovelace", "101 Main St");
        IO.println(p.getName());
        p.setAddress("5 Elm St");
        IO.println(p.getAddress());
    }
}

class Person {
    private String name;
    private String address;

    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    public String getName() { return name; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
}
```

This is just a normal class so far, using what we learned.

But now we've been assigned to write a Student class.

How much of the above would you copy-paste?

Example: Student

Now we need a Student

A `Student` is a person plus classes and grades. First attempt:

```
class Main {
    void main() {
        Student s = new Student("Ada Lovelace", "101 Main St");
        IO.println(s.getName());
    }
}

class Student {
    private String name;                    // <- same as Person
    private String address;                 // <- same as Person
    private String[] classes;               //    new
    private String[] grades;                //    new

    public Student(String name, String address) {
        this.name = name;                   // <- same as Person
        this.address = address;             // <- same as Person
    }

    public String getName() { return name; }                            // <- same a
    public String getAddress() { return address; }                      // <- same a
    public void setAddress(String address) { this.address = address; }  // <- same a
    public String[] getClasses() { return classes; }                    //    new
    // ...
}
```

Most of this class is a photocopy of `Person`.

And `Professor` (rank, salary) would photocopy it again.

What do we do when we want to add a new field or change how names are stored?

Person Subclass demo

`extends`: keep only the differences

(There is nothing to run or submit here.)

These three files show what it looks like to keep separate classes that each repeat all of `Person`. `Person`, `Staff` and `Student` overlap almost entirely.

We will refactor them together so that each class keeps only the parts that are its own.

- `extends Person`: `Student` **inherits** every field and method of `Person`.
- The subclass contains only the differences: new fields, its own constructor, and new methods.
- `s.getName()` works on a `Student` even though `Student` never declares it.

Let's apply this to the code together. (The final version is on the next slide.)

Refactoring the Example

The class Person has fields name, address, as well as accessors and mutators

```
public class Person {
    private String name;
    private String address;

    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }
}
```

The class Student “extends” Person

- Inherits the fields and methods from Person
- Adds classes and grades (and more accessors and mutators)

```
public class Student extends Person {
    private String[] classes;
    private String[] grades;

    public Student(String name, String address, String[] classes, String[] grades) {
        super(name, address);
        this.classes = classes;
        this.grades = grades;
    }

    public String[] getClasses() {
        return classes;
    }

    public void setClasses(String[] classes) {
        this.classes = classes;
    }

    // and more…
}
```

The class Staff “extends” Person

- Inherits the fields and methods from Person
- Adds title and salary (and more accessors and mutators)

```
public class Staff extends Person {
    private String jobTitle;
    private double salary;

    public Staff(String name, String address, String jobTitle, double salary) {
        super(name, address);
        this.jobTitle = jobTitle;
        this.salary = salary;
    }

    public String getJobTitle() {
        return jobTitle;
    }

    public void setJobTitle(String title) {
        this.jobTitle = title;
    }

    // and more…
}
```

Common fields and methods go in Person, and are inherited by its subclasses

Class-specific fields and methods go in their respective class

Demo: Press Run: the inherited `getName()` works on both subclasses.

```
class Main {
    void main() {
        Student s = new Student("Ada Lovelace", "101 Main St", new String[] {"CS 180"}
        Staff k = new Staff("Donald Knuth", "3 Stanford Ave", "Registrar", 62000);
        IO.println(s.getName() + " takes " + s.getClasses()[0]);   // getName() is inh
        IO.println(k.getName() + " is the " + k.getJobTitle());    // so is getName()
    }
}

class Person {
    private String name;
    private String address;

    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    public String getName() { return name; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
}

class Student extends Person {
    private String[] classes;
    private String[] grades;

    public Student(String name, String address, String[] classes, String[] grades) {
        super(name, address);
        this.classes = classes;
        this.grades = grades;
    }

    public String[] getClasses() { return classes; }
    public void setClasses(String[] classes) { this.classes = classes; }
}

class Staff extends Person {
    private String jobTitle;
    private double salary;
```

```
    public Staff(String name, String address, String jobTitle, double salary) {
        super(name, address);
        this.jobTitle = jobTitle;
        this.salary = salary;
    }

    public String getJobTitle() { return jobTitle; }
    public void setJobTitle(String title) { this.jobTitle = title; }
}
```

Inheritance

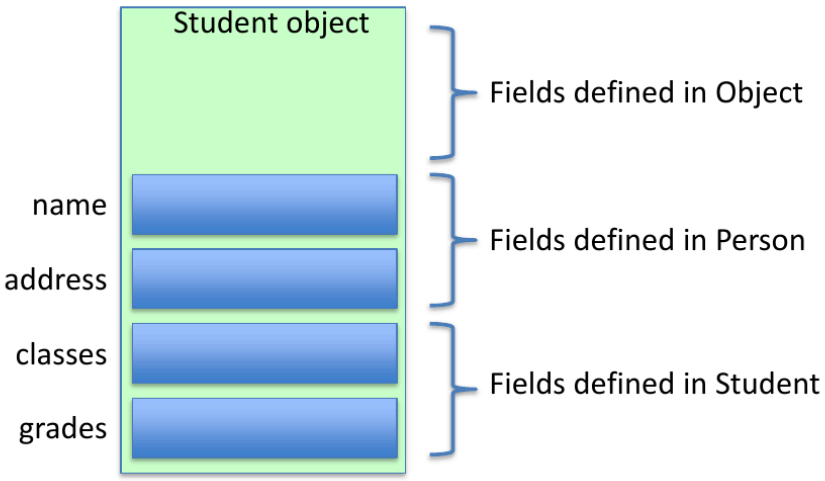
One object

Rather than duplicating members (fields and methods) among these classes, Java allows classes to share member definitions in a hierarchical fashion

One class can “extend” another “inheriting” fields and methods from it

Terminology: the “subclass” *inherits* from the “superclass”.

A `Student` object physically contains its inherited fields:



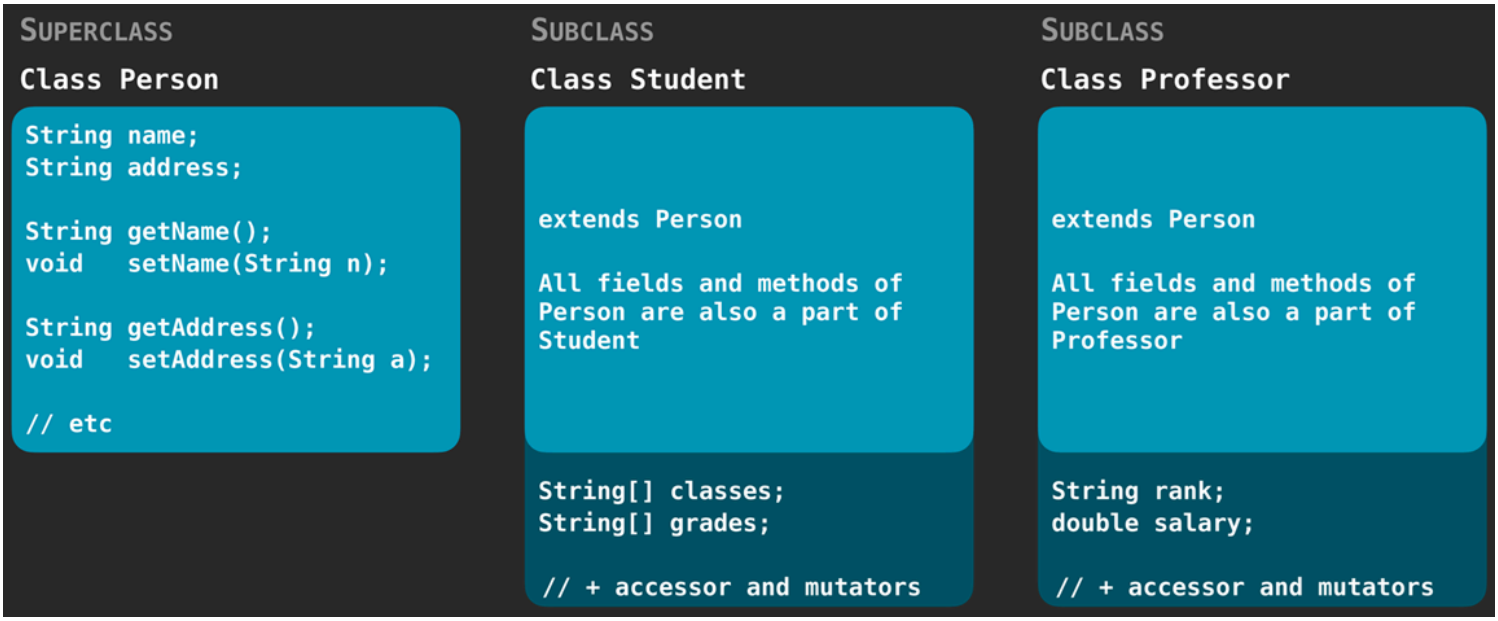
When we call `new Student("Michael", ...)` there is only one object created.

The `Person` parts and `Student` parts live together.

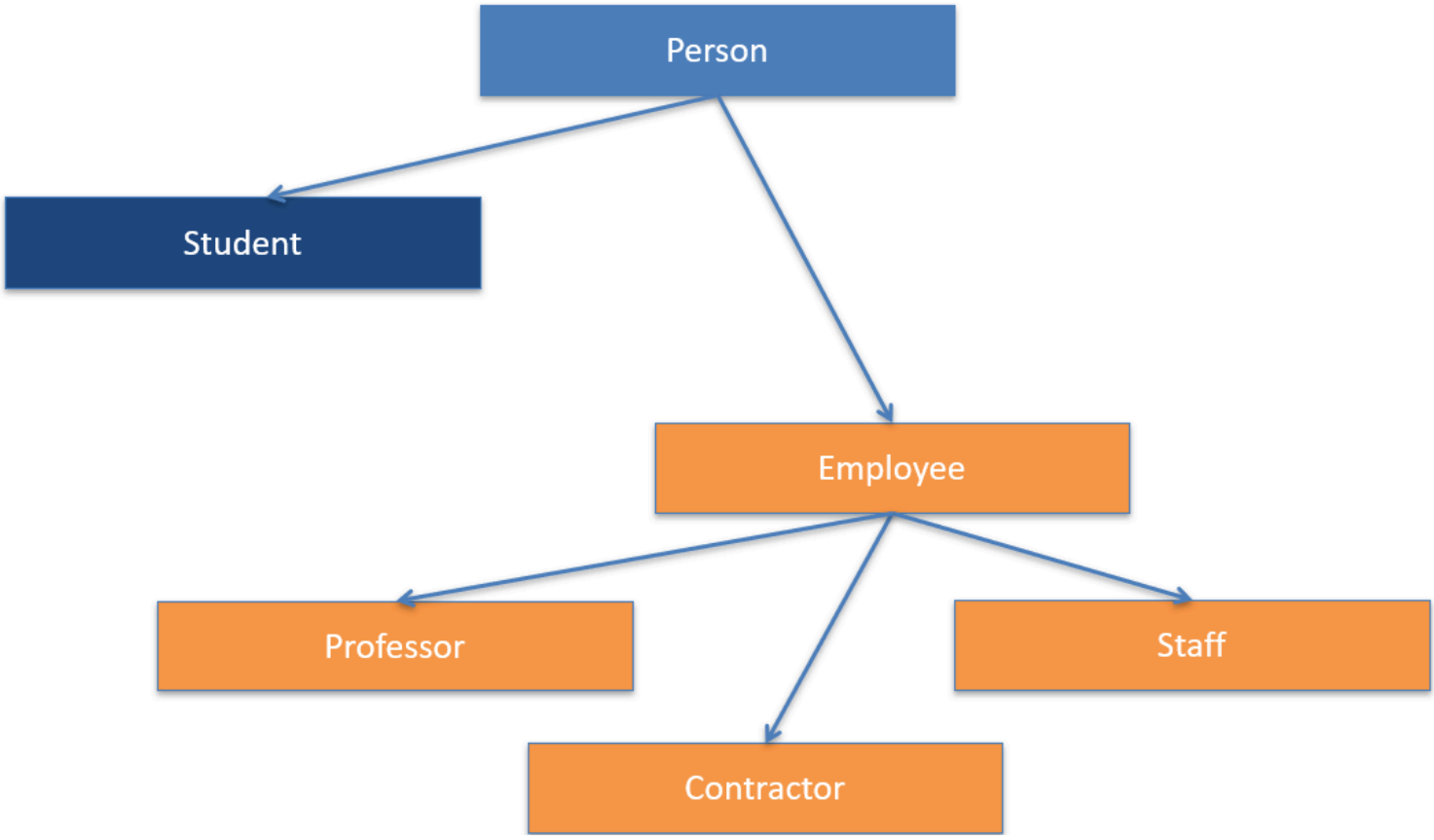
This is why it works to call `getName()` on a `Student`

The family, drawn two ways

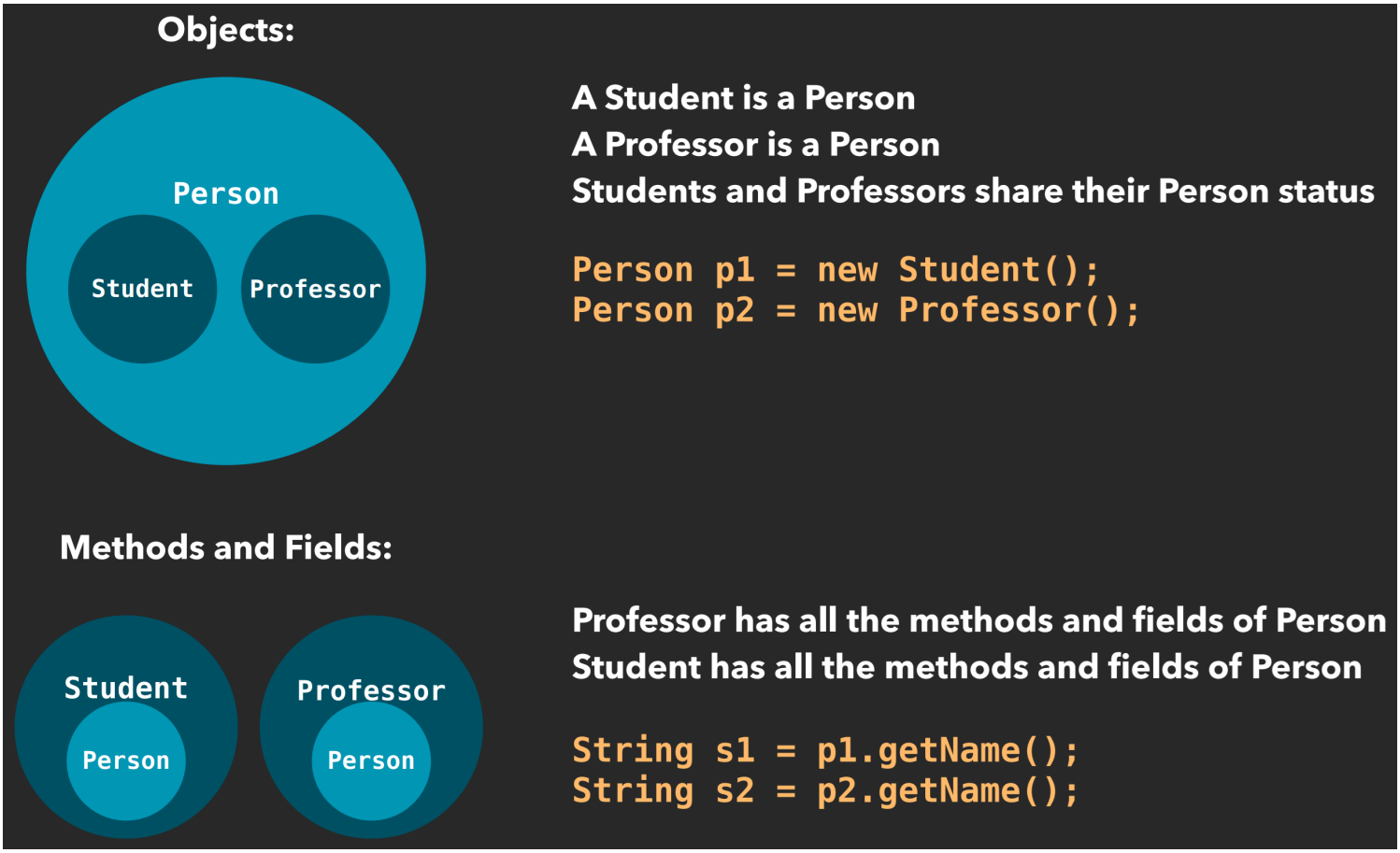
First, let's visualize the code:



We extend this to make a multi-level hierarchy as complex as we need:



The subset picture



Every `Student` **is a** `Person`. So this line is legal:

```
Person p1 = new Student(...);    // a Person-typed variable, holding a Student
```

Anywhere a `Person` is expected or needed, a `Student` qualifies.

This line does NOT compile: `Student s = new Person(...);`

- not every Person is a Student.
- The subset relation only points one way.

Constructor Woes

Watch this fail: the constructor that wasn't there

`Wheel` has a constructor. `Tire` extends `Wheel` , but we're forgetting something....

```
class Main {
    void main() {
        Tire t = new Tire(17.5, 2.0);
        IO.println("built a tire");
    }
}

class Wheel {
    private double radius;

    public Wheel(double radius) {
        this.radius = radius;
    }
}

class Tire extends Wheel {
    private double width;

    public Tire(double radius, double width) {
        this.width = width;          // ...where does radius go?
    }
}
```

We get this rather cryptic error:

```
error: constructor Wheel in class Wheel cannot be applied to given types;
    public Tire(double radius, double width) {
                                             ^
  required: double
```

What did we forget? The `super(radius)` that we used in `Student`

Java secretly inserted a call to `super()` , which is a **zero-argument** `Wheel` constructor.

`Wheel` doesn't have one, everything explodes.

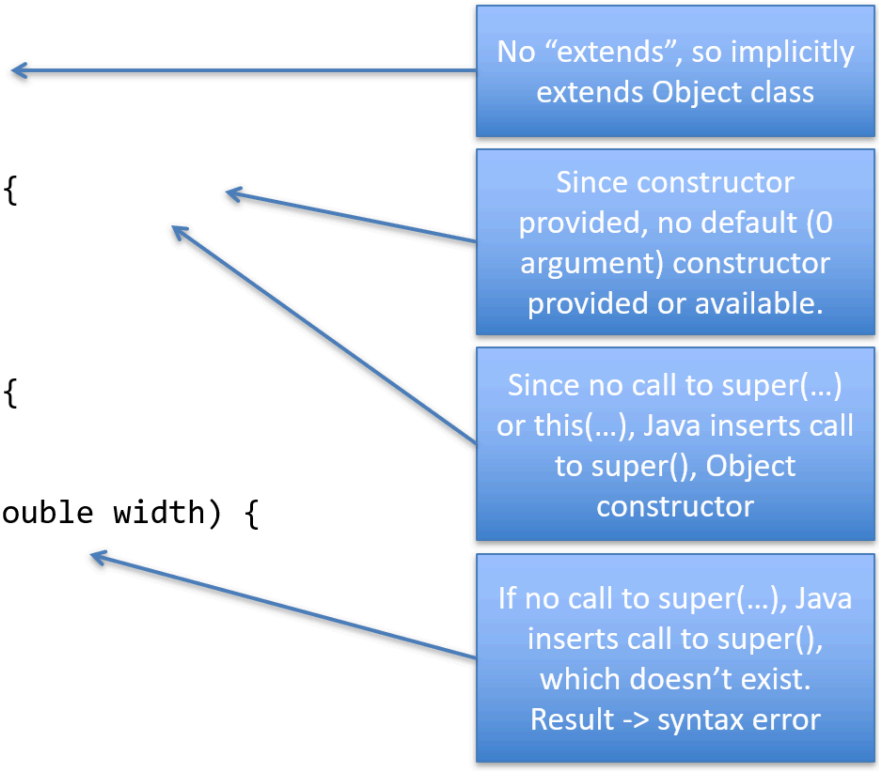
The compiler error is so cryptic because it can't show you the invisible line causing the error.

```
public class Wheel {
    private double radius;

    public Wheel(double radius) {
        this.radius = radius;
    }
}

public class Tire extends Wheel {
    private double width;

    public Tire(double radius, double width) {
        // super(radius);
        this.width = width;
    }
}
```



Chaining Rules

The chaining rules

When a `Student` is born, the `Person` parts must be initialized too by one of `Person`'s constructors.

Java **enforces** this:

1. Every constructor begins by calling another constructor:
    1. either **`this(...)`** (same class) or
    2. **`super(...)`** (superclass).

2. If you write neither, Java inserts **`super()`** , the zero-argument superclass constructor.

3. So if the superclass has no zero-argument constructor, you **must** write `super(...)` yourself with arguments it can accept.

Fix for `Tire`:

```
class Main {
    void main() {
        Tire t = new Tire(17.5, 2.0);
        IO.println(t.getRadius() + " by " + t.getWidth());
    }
}

class Wheel {
    private double radius;

    public Wheel(double radius) {
        this.radius = radius;
    }

    public double getRadius() { return radius; }
}

class Tire extends Wheel {
    private double width;

    public Tire(double radius, double width) {
        super(radius);              // initialize the Wheel part first
        this.width = width;
    }

    public double getWidth() { return width; }
}
```

This is called *constructor chaining*.

When constructing an object of some class, it is important that all the constructors up the inheritance chain have an opportunity to initialize the object under construction.

Why not just set the field yourself?

Could `Tire` skip `super(radius)` and write this instead?

```
class Main {
    void main() {
        Tire t = new Tire(17.5, 2.0);
    }
}

class Wheel {
    private double radius;

    public Wheel() { this(17.5); }           // a zero-argument constructor, so the in
    public Wheel(double radius) {
        this.radius = radius;
    }
}

class Tire extends Wheel {
    private double width;

    public Tire(double radius, double width) {
        this.radius = radius;        // error: radius has private access in Wheel
        this.width = width;
    }
}
```

No: `radius` is `private` to `Wheel`.

A subclass *has* the field but **cannot touch it**.

The superclass's constructor (and its accessors/mutators) are the doors.

`private` stays private, even from your own children.

When the parent has several constructors

`super(...)` picks among the parent's constructors by matching argument types

```
class Main {
    void main() {
        Tire standard = new Tire(2.0);          // chains to Wheel()
        Tire custom = new Tire(20.0, 2.5);      // chains to Wheel(double)
        IO.println(standard.getRadius());
        IO.println(custom.getRadius());
    }
}

class Wheel {
    private double radius;

    public Wheel() { this(17.5); }        // parent now has TWO constructors
    public Wheel(double radius) { this.radius = radius; }

    public double getRadius() { return radius; }
}

class Tire extends Wheel {
    private double width;

    public Tire(double width) {
        super();                // chains to Wheel()
        this.width = width;
    }

    public Tire(double radius, double width) {
        super(radius);          // chains to Wheel(double)
        this.width = width;
    }
}
```

Pause and predict (poll)

Q. `Parent` has only this constructor: `Parent(int x)`. What happens?

```
class Child extends Parent {
    Child() {
        // nothing here
    }
}
```

- **A.** Compile error
- **B.** It runs; Parent's fields get default values
- **C.** It throws an exception when a Child is created

- **D.** Child automatically inherits Parent's (int) constructor

Overriding

Overriding: same signature, new behavior

`Person` has a perfectly good `introduce()`. But students don't talk like the registrar:

```
class Main {
    void main() {
        Person p = new Person("Ada");
        Student s = new Student("Ada");
        p.introduce();
        s.introduce();
    }
}

class Person {
    private String name;

    Person(String name) { this.name = name; }

    String getName() { return name; }

    void introduce() { IO.println("Hi, I'm " + name); }
}

class Student extends Person {
    Student(String name) { super(name); }

    @Override
    void introduce() { IO.println("Hey, I'm " + getName() + ", CS major"); }
}
```

A subclass method with the same signature **overrides** the inherited one:

- on a `Student`, the `Student` version runs.
- Not overridden? You inherit the original. (That's most methods, most of the time.)

`@Override`: a typo shield

`@Override` asks the compiler to check that you're really overriding:

```
class Main {
    void main() {
        new Typo().greet();
    }
}

class Base {
    void greet() { IO.println("hello"); }
}

class Typo extends Base {
    @Override
    void gret() { IO.println("hi"); }        // typo!
}
```

```
error: method does not override or implement a method from a supertype
```

Without `@Override`:

- no error: `gret()` is quietly a **brand-new method**,
- `greet()` is quietly inherited unchanged,
- and your "override" never runs.

With the annotation, the bug turns into a compiler error.

super.method

Keep the parent's work: `super.method()`

Sometimes the subclass wants the parent's behavior *plus more*, not a replacement.

Remember: extend, don't repeat:

```
class Main {
    void main() {
        new Student("Ada").introduce();
    }
}

class Person {
    private String name;

    Person(String name) { this.name = name; }

    void introduce() { IO.println("Hi, I'm " + name); }
}

class Student extends Person {
    Student(String name) { super(name); }

    @Override
    void introduce() {
        super.introduce();                      // parent's version first...
        IO.println("...and I'm a CS major");
    }
}
```

```
Hi, I'm Ada
...and I'm a CS major
```

`super.introduce()` = "run the version I overrode."

It works for return values, too

The parent's answer is a value you can **build on**. Appending to a String:

```
class Main {
    void main() {
        Student s = new Student("Ada");
        IO.println(s);
    }
}

class Person {
    private String name;

    Person(String name) { this.name = name; }

    public String toString() { return "Person " + name; }
}

class Student extends Person {
    Student(String name) { super(name); }

    @Override
    public String toString() {
        return super.toString() + ", studying CS";   // parent's answer + more
    }
}
```

or adding a number:

```
class Main {
    void main() {
        IO.println(new Employee(1000).getWeeklyPay());      // 1000.0
        IO.println(new Manager(1000, 250).getWeeklyPay());  // 1250.0
    }
}

class Employee {
    private double weeklyPay;

    Employee(double weeklyPay) { this.weeklyPay = weeklyPay; }

    public double getWeeklyPay() { return weeklyPay; }
}

class Manager extends Employee {
    private double bonus;

    Manager(double weeklyPay, double bonus) {
        super(weeklyPay);
        this.bonus = bonus;
    }

    @Override
    public double getWeeklyPay() {
        return super.getWeeklyPay() + bonus;   // build on the parent's answer
    }
}
```

```
new Employee(1000).getWeeklyPay()    // 1000.0
new Manager(1000, 250).getWeeklyPay() // 1250.0
```

The pattern: **call the super version, then adjust its answer.**

Overloading vs overriding

	Overloading	Overriding
Where	**same class**	superclass --> subclass
Name	same	same
Signature	**different** parameters	**identical** signature
Chosen by	argument types, at compile time	the **object**, at run time

Pause and predict (poll)

Predict: whose method runs?

```
Person p = new Student("Ada");
p.introduce();
```

The **variable** says `Person`. The **object** is a `Student`.

```
public class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
    void introduce() { IO.println("Hi, I'm " + name); }
}

class Student extends Person {
    Student(String name) { super(name); }

    @Override
    void introduce() { IO.println("Hey, I'm " + getName() + ", CS major"); }

    @Override
    public String toString() { return "Student(" + getName() + ")"; }

    public static void main() {
        Person p = new Student("Ada");
        p.introduce();
    }
}
```

Q. Whose `introduce()` runs?

- **A.** `Hi, I'm Ada` : the variable's type decides
- **B.** `Hey, I'm Ada, CS major` : the object decides
- **C.** compile error: you can't call it through a `Person` variable

Dynamic Binding

The mixed room

Because every `Student` and `Staff` *is a* `Person`, one array holds the whole room:

```
class Main {
    void main() {
        Person[] room = { new Student("Ada"), new Staff("Knuth"), new Person("Sam") };
        for (Person q : room) {
            q.introduce();
        }
    }
}

class Person {
    private String name;

    Person(String name) { this.name = name; }

    String getName() { return name; }

    void introduce() { IO.println("Hi, I'm " + name); }
}

class Student extends Person {
    Student(String name) { super(name); }

    @Override
    void introduce() { IO.println("Hey, I'm " + getName() + ", CS major"); }
}

class Staff extends Person {
    Staff(String name) { super(name); }

    @Override
    void introduce() { IO.println(getName() + " here, Registrar's office."); }
}
```

```
Hey, I'm Ada, CS major
Knuth here, Registrar's office.
Hi, I'm Sam
```

One declared type, but three different behaviors.

Each object answers as what it really is.

This has a name: **polymorphism** ("many shapes"). One piece of code, many behaviors.

Why this matters

The loop never changes again.

- Add a `Dean` class tomorrow.
  - `extends Person`, override `introduce()`
  - and the same loop greets deans.
  - Zero edits to the loop.

- Code written against the **supertype** is finished;

- growth happens by **adding subclasses**, not editing callers.

Game Night Returns: the Loaded Die

Game night: the loaded die

On Friday we wrote `Die` and `DiceCup`.

A loaded die *is a* die, but it has strong opinions about how its rolls should go:

```java
import java.util.Random;

class Main {
    void main() {
        LoadedDie cheat = new LoadedDie();
        for (int i = 0; i < 5; i++) {
            IO.print(cheat.roll() + " ");     // the house always wins
        }
        IO.println();
    }
}

class Die {
    private Random rng;
    private int numFaces;
    private int state;

    public Die(int numFaces) {
        this.rng = new Random();
        this.numFaces = numFaces;
        this.state = 0;
    }

    public int roll() {
        state = 1 + rng.nextInt(numFaces);
        return state;
    }

    public int getState() { return state; }
}

class LoadedDie extends Die {

    public LoadedDie() {
        super(6);                 // a loaded die is still a six-sided die
    }

    @Override
    public int roll() {
        int r = super.roll();     // roll it for real...
        while (r != 6) {
            r = super.roll();     // ...and keep rolling until the house likes it
        }
        return r;
    }
}
```

Notice what `LoadedDie` could **not** do:

- `state` and `numFaces` are `private` in `Die` .
- The cheat works within the rules: it calls `super.roll()` (which updates the die's state honestly) until the answer is a 6.

We can drop a `LoadedDie` into a `DiceCup` without ever touching `DiceCup`

```
import java.util.Random;

class Main {
    void main() {
        DiceCup cup = new DiceCup(3);
        cup.addDie(new Die(6));
        cup.addDie(new Die(6));
        cup.addDie(new LoadedDie());   // a LoadedDie IS a Die
        IO.println("total: " + cup.rollAll());
    }
}

class Die {
    private Random rng;
    private int numFaces;
    private int state;

    public Die(int numFaces) {
        this.rng = new Random();
        this.numFaces = numFaces;
        this.state = 0;
    }

    public int roll() {
        state = 1 + rng.nextInt(numFaces);
        return state;
    }
}

class LoadedDie extends Die {
    public LoadedDie() { super(6); }

    @Override
    public int roll() {
        int r = super.roll();
        while (r != 6) {
```

```
            r = super.roll();
        }
        return r;
    }
}

class DiceCup {                     // Friday's cup, untouched: it knows only Die
    private Die[] dice;
    private int count;

    public DiceCup(int capacity) {
        dice = new Die[capacity];
        count = 0;
    }

    public boolean addDie(Die d) {
        if (count == dice.length) {
            return false;
        }
        dice[count] = d;
        count++;
        return true;
    }

    public int rollAll() {
        int total = 0;
        for (int i = 0; i < count; i++) {
            total += dice[i].roll();   // each die answers as what it really is
        }
        return total;
    }
}
```

Notes:

- `rollAll()` was written against `Die`, before `LoadedDie` existed.
- `rollAll()` loops, calls `roll()` on each element and the loaded die answers **as what it really is**.

That's **polymorphism** in action.

Pause and predict (poll)

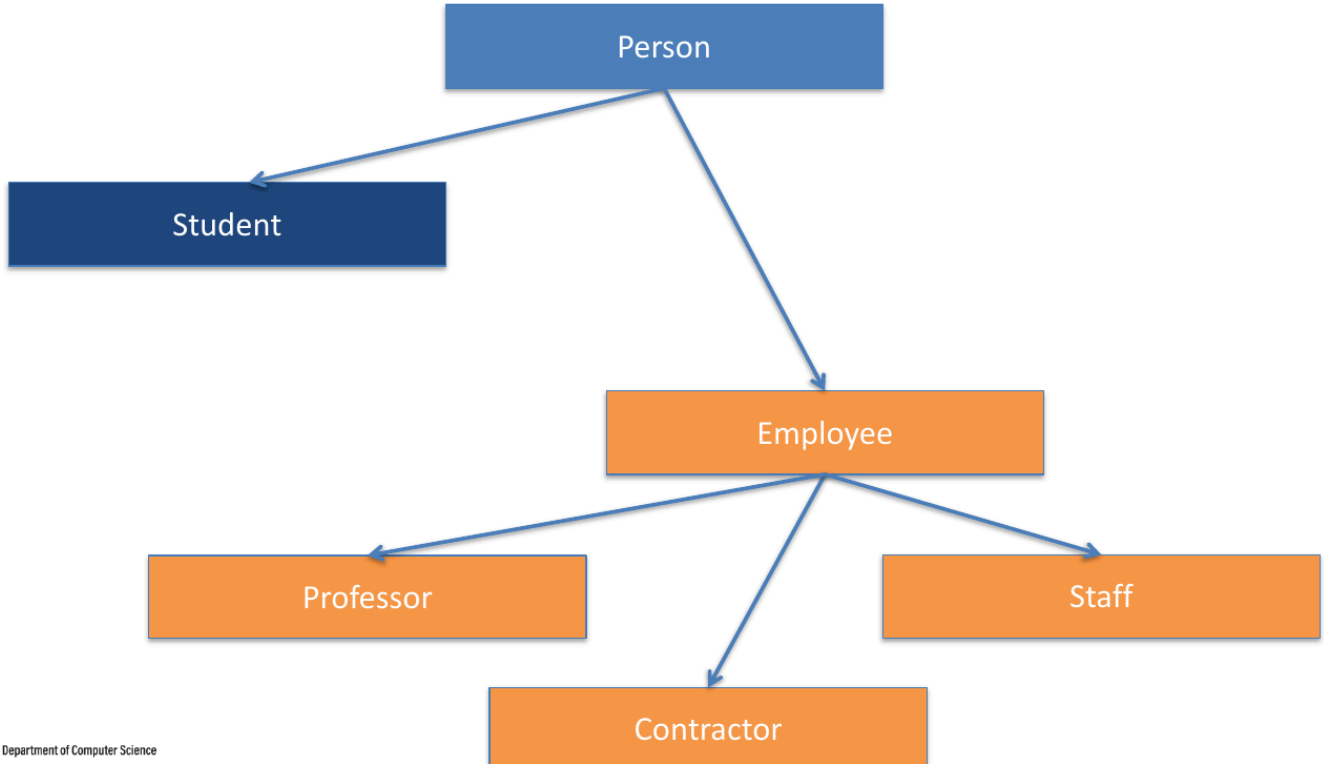
Q. `DiceCup` (written before `LoadedDie` existed) holds two `Die` and one `LoadedDie`. What does `cup.rollAll()` do?

- **A.** Compile error: addDie takes Die, not LoadedDie
- **B.** It works, and the loaded die reports its loaded roll
- **C.** It works only if DiceCup is edited to know about LoadedDie
- **D.** It works, but the LoadedDie behaves like a normal Die inside the cup

Object

`Object`: the root of everything

- A class with no `extends` implicitly extends `Object`
- So *every* class descends from `Object`
- one tree, one root
- A class gets exactly **one** `extends` (Java is *single inheritance*)



`Object` gives every class some standard methods.

We'll look at one: `public String toString()`

Card: final Fields and toString

(This slide was originally posted last week, but not covered)

Game night, part two: a playing card

Idea 1: `final` fields

A card never changes what it is.

Mark the fields `final` and Java enforces it:

- `final` fields must be assigned **exactly once, in the constructor**,
- and can never change after:

```
class Main {
    void main() {
        Card c = new Card("Ace", "Spades");
        IO.println(c.getRank() + " of " + c.getSuit());
        IO.println(c);                      // predict!
    }
}

class Card {
    private final String rank;   // "Ace", "2", ..., "King"
    private final String suit;   // "Hearts", "Spades", ...

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String getRank() {
        return rank;
    }

    public String getSuit() {
        return suit;
    }
}
```

A card cannot change, so we have getters, but no setters.

An object like this is **immutable**, like `String`.

Compare the Die: `roll()` *mutates* its state. Both designs are correct for their objects.

Note: a setter on a final field is a compile error:

```
public void setRank(String r) {
    this.rank = r;      // COMPILE ERROR: cannot assign a value to final variable rank
}
```

Idea 2: what does printing an object do?

Run the snippet above and look at its last line.

`IO.println(c)` printed something like `Card@7344699f`: the class name and a value called a hash.

- This is Java's default description of an object, and it is useless usually.
- The fix: give the class a `toString()` method.
- `println` and String concatenation call `toString()` automatically:

```
class Main {
    void main() {
        Card c = new Card("Ace", "Spades");
        IO.println(c);                    // Ace of Spades
        IO.println("drew: " + c);         // drew: Ace of Spades
    }
}

class Card {
    private final String rank;
    private final String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() {
        return rank + " of " + suit;
    }
}
```

`toString` is the object's self-description. Every class you write should have one.

toString: Always an Override

`toString`: always an Override

You may have tried printing an object at some point:

```
class Main {
    void main() {
        Person plain = new Person("Sam", "101 Main St");
        IO.println(plain);            // Person@19d37183, or some such
    }
}

class Person {
    private String name;
    private String address;

    Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    String getName() { return name; }
}
```

What is that output?

`Person@19d37183` is **`Object`'s** `toString()` method, and every class inherits it from the root.

When you wrote your own `toString()` methods, you will be overriding an inherited method.

```
class Main {
    void main() {
        Student s = new Student("Ada", "101 Main St");
        IO.println(s);
    }
}

class Person {
    private String name;
    private String address;

    Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    String getName() { return name; }
}

class Student extends Person {
    Student(String name, String address) { super(name, address); }

    @Override
    public String toString() {
        return "Student(" + getName() + ")";
    }
}
```

```
Student(Ada)
```

Terminology

Terminology (all five aliases)

- `Student` **extends** `Person`:
    - `Student` = **subclass** = child class = derived class
    - `Person` = **superclass** = parent class = base class
- Sub/super feels backwards?
    - The **subclass has more stuff**.
    - Think **sets**, not size:
    - the set of Students is a **subset** of the set of Persons
    - every Student is a Person, not vice versa.

Pause and predict

Q. Which assignment compiles?

- **A.** `Person p = new Student(...);`
- **B.** `Student s = new Person(...);`
- **C.** Both
- **D.** Neither

The Classic main

The classic `main`

```
public static void main(String[] args) { … }
```

word	meaning
`public`	any code anywhere may call it: the Java launcher lives outside your class
`static`	belongs to the class, not to an object: when the program starts, **no object exists yet**, so the launcher can only call a static method
`void`	returns nothing
`main`	the name the launcher looks for
`String[] args`	an array of the words typed after the program name on the command line

```
class Main {
    public static void main(String[] args) {
        IO.println("classic main: args.length = " + args.length);
        for (int i = 0; i < args.length; i++) {
            IO.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

Run as `java Main add 5 17.25`, this prints `args.length = 3`, then `add`, `5` and `17.25`.

Everything in `args` is a String: `args[1]` is `"5"`, not `5`.

Pressing Run on a slide supplies no arguments, so `args.length` is 0.

Our `void main()` is the same method without the boilerplate:

- Java builds an object of your class first and calls `main` on it
- That's why `this.` has been available all semester.

- Both forms are correct.

Extent (extra reading)

Extra reading: not presented in class

Scope vs. extent (lifetime)

- **Scope:** *where* is the name usable? (a compile-time region)
- **Extent:** *how long* does the value live? (a run-time lifetime)

For a value stored in a **local variable**, extent = scope:

- it is born at its declaration
- and dies at the block's `}`:

```
class Main {
    void main() {
        int pass = 0;
        while (pass < 3) {
            int i = 0;      // re-created and re-initialized EACH pass
            i++;
            IO.println(i);  // always prints 1
            pass++;
        }
    }
}
```

Object extent: alive while reachable

An **object** lives as long as *something* still references it.

When the last reference is gone, it becomes eligible for **garbage collection**:

```
String s = new String("Purdue");    // object P
String t = s;                       // s and t both point at P
s = new String("Indiana State");    // s → new object; P still held by t
t = new String("Ball State");       // t → new object; nothing holds P now
```

```
after line 2:   s ──► "Purdue" ◄── t
after line 3:   s ──► "Indiana State"      t ──► "Purdue"
after line 4:   s ──► "Indiana State"      t ──► "Ball State"
```

`"Purdue"` is now unreachable and eligible for garbage collection.

You never delete objects by hand in Java; you just stop pointing at them.

Take Home: Staff

`Person.java` is given with a `toString()` that describes a person as `Ada Lovelace (101 Main St)`. Do not change it.

Finish **`Staff`**, a `Person` who works here. The fields are declared; the methods are stubs.

- **`Staff(String name, String address, String jobTitle, double salary)`**: build the `Person` part first, then store the two new fields. As shipped this constructor does not compile. Press **Run**, read the error, and recognize it from today's lecture before you fix it.
- **`getJobTitle()`** and **`getSalary()`**.
- **`giveRaise(double percent)`**: raise the salary by that percentage of its current value. A 10 percent raise on 50000 gives 55000; a second 10 percent raise gives 60500.
- **`toString()`**: the `Person` description, then `, ` and the job title: `Ada Lovelace (101 Main St), Registrar`. Build on `super.toString()` rather than rebuilding the name and address yourself.

`Main.java` uses your class through a `Staff` variable and then through a `Person` variable; the last line prints the `Staff` description either way, because the object decides.

Run runs `Main`; press **Test** to run the tests.